

Kore

Videojuego 2D de Acción y Aventura

Documentación del Software

Trabajo Final de Semestre

Herramientas de Desarrollo de Videojuegos

Motor de juego:	Unity 6000.4.6f1
Lenguaje:	C# (.NET / Mono)
Pipeline:	Universal Render Pipeline (URP)
Plataformas:	Windows / Web
Fecha:	Junio 2026

Autores:

Ángel Gabriel Burbano

Camilo Ramírez

Índice

1. Descripción General del Proyecto	2
2. Arquitectura del Software	2
2.1. Tecnologías Utilizadas	2
2.2. Estructura de Escenas	3
2.3. Capas del Proyecto (Layers)	3
3. Scripts Principales del Juego	3
3.1. PlayerController – Control del Jugador	3
3.2. Enemy – Inteligencia Artificial de Enemigos	5
3.3. ScoreManager – Gestión de Puntuación	6
3.4. DialogueManager – Sistema de Diálogos	7
3.5. HealthBar – Barra de Vida	7
3.6. ControlesMoviles – Joystick Virtual	8
3.7. CameraController – Seguimiento de Cámara	9
3.8. VolumeController – Control de Audio	9
4. Mecánicas de Juego	10
4.1. Sistema de Combate	10
4.2. Sistema de Vida y Escudo	10
4.3. Recolectables y Puntuación	10
4.4. Enemigos	10
4.5. Trampas y Peligros	11
5. Interfaz de Usuario (UI)	11
6. Flujo del Juego	11
7. Conclusiones	11

1. Descripción General del Proyecto

Kore es un videojuego 2D de acción y aventura desarrollado en Unity como proyecto final de semestre. El juego presenta a un personaje principal que debe recorrer múltiples niveles, enfrentarse a enemigos, derrotar a un jefe final (*boss*) y recolectar objetos para acumular puntuación.

El proyecto integra mecánicas clásicas del género plataformero-acción tales como:

- Sistema de salud y escudo del jugador.
- Combate cuerpo a cuerpo con espada y ataque a distancia.
- Enemigos con inteligencia artificial de patrullaje y persecución.
- Recolección de monedas, gemas y estrellas para la puntuación.
- Diálogos con personajes NPC.
- Soporte de controles táctiles mediante joystick virtual (para dispositivos móviles).
- Múltiples escenas: menú principal, niveles de juego y pantallas de victoria/derrota.

2. Arquitectura del Software

2.1. Tecnologías Utilizadas

Tecnología	Descripción
Unity 6000.4.6f1	Motor de juego principal. Gestiona el ciclo de vida, física, renderizado y audio.
C# / Mono	Lenguaje de scripting para toda la lógica del juego.
Universal Render Pipeline (URP)	Pipeline de renderizado optimizado para 2D/3D ligero.
TextMeshPro	Sistema de renderizado de texto de alta calidad para la UI.
Unity.Postprocessing	Efectos visuales de post-procesado (bloom, color grading).
Unity.Timeline	Sistema de cinemáticas y secuencias animadas.
Physics2D	Motor de física 2D de Unity para colisiones y movimiento.
SceneManager	Gestión de carga y transición entre escenas.
PlayerPrefs	Almacenamiento de datos persistentes (puntuación, configuración).

Cuadro 1: Stack tecnológico del proyecto

2.2. Estructura de Escenas

El juego está organizado en **5 escenas** (niveles), cada una cargada secuencialmente a través del SceneManager:

- **Level 0 (Menú Principal):** Pantalla de inicio con opciones de jugar, configuración de volumen y salida.
- **Level 1 – 3 (Niveles de Juego):** Escenas de juego con tilemap, enemigos, objetos coleccionables, trampas y diálogos.
- **Level 4 (Boss / Final):** Escena del jefe final con mecánicas de combate extendidas.

2.3. Capas del Proyecto (Layers)

Las capas de Unity utilizadas para gestionar colisiones y detección son:

- **Ground** – Superficies transitables.
- **Player** – Capa del jugador.
- **Enemies / Patrols** – Capa de enemigos.
- **Water** – Zonas de agua/peligro.
- **Background** – Fondos no colisionables.
- **ZonasOscuras** – Zonas con efectos de oscuridad.

3. Scripts Principales del Juego

A continuación se describen los scripts C# más importantes del proyecto, extraídos del ensamblado Assembly-CSharp.dll.

3.1. PlayerController – Control del Jugador

Este es el script central del jugador. Maneja el movimiento, salto, ataque y la interacción con el entorno.

```
1 using UnityEngine;
2
3 public class PlayerController : MonoBehaviour
4 {
5     [SerializeField] private float speed;
6     [SerializeField] private float saltar;
7     [SerializeField] private LayerMask whatIsGround;
8     [SerializeField] private float attackRange;
9     [SerializeField] private float attackDamage;
10
11     private Rigidbody2D rb;
12     private Animator animator;
13     private bool grounded;
```

```
14 private bool isFlipped;
15 private float nextAttactTime;
16
17 // Sistema de vida y escudo
18 private float setHealth;
19 private float shield;
20 private float shieldValue;
21
22 void FixedUpdate()
23 {
24     // Movimiento horizontal
25     float moveX = Input.GetAxis("Horizontal");
26     rb.velocity = new Vector2(moveX * speed, rb.velocity.y);
27
28     // Animacion de movimiento
29     animator.SetFloat("Velocidad", Mathf.Abs(moveX));
30
31     // Voltear el sprite segun la direccion
32     if (moveX > 0 && isFlipped) Flip();
33     else if (moveX < 0 && !isFlipped) Flip();
34 }
35
36 public void TakeDamage(float damage)
37 {
38     if (shield > 0)
39         shield -= damage * shieldValue;
40     else
41         setHealth -= damage;
42
43     if (setHealth <= 0)
44         Die();
45 }
46
47 private void Flip()
48 {
49     isFlipped = !isFlipped;
50     transform.Rotate(0f, 180f, 0f);
51 }
52
53 public void ButtonJump()
54 {
55     if (grounded)
56         rb.AddForce(Vector2.up * saltar, ForceMode2D.Impulse);
57 }
58
59 public void ButtonAttack()
60 {
61     if (Time.time >= nextAttactTime)
62     {
63         animator.SetTrigger("Attack");
64         Collider2D[] hits = Physics2D.OverlapCircleAll(
65             attackPoint.position, attackRange, enemyLayers);
66         foreach (Collider2D hit in hits)
67             hit.GetComponent<Enemy>().TakeDamage(attackDamage);
68
69         nextAttactTime = Time.time + 1f / attactRate;
70     }
71 }
```

72 }

Listing 1: PlayerController.cs – Estructura base

3.2. Enemy – Inteligencia Artificial de Enemigos

Controla el comportamiento de los enemigos: patrullaje, detección y ataque al jugador.

```
1 using UnityEngine;
2
3 public class Enemy : MonoBehaviour
4 {
5     [SerializeField] private float speed;
6     [SerializeField] private float range;
7     [SerializeField] private float attackDamage;
8     [SerializeField] private float takeDamage;
9
10    private Transform player;
11    private Animator animator;
12
13    void Update()
14    {
15        float distance = Vector2.Distance(
16            transform.position, player.position);
17
18        if (distance <= range)
19        {
20            LookAtPlayer();
21            animator.SetBool("Chase", true);
22            MoveTowards();
23        }
24        else
25        {
26            Patrol();
27            animator.SetBool("Chase", false);
28        }
29    }
30
31    private void LookAtPlayer()
32    {
33        if (player.position.x < transform.position.x)
34            transform.eulerAngles = new Vector3(0, 180, 0);
35        else
36            transform.eulerAngles = Vector3.zero;
37    }
38
39    public void TakeDamage(float damage)
40    {
41        takeDamage -= damage;
42        if (takeDamage <= 0) Die();
43    }
44 }
```

Listing 2: Enemy.cs – IA de enemigos

3.3. ScoreManager – Gestión de Puntuación

Gestiona los tres tipos de coleccionables y calcula el puntaje total.

```
1 using UnityEngine;
2 using TMPro;
3
4 public class ScoreManager : MonoBehaviour
5 {
6     public static ScoreManager instance;
7
8     [SerializeField] private TextMeshProUGUI textCoins;
9     [SerializeField] private TextMeshProUGUI textGems;
10    [SerializeField] private TextMeshProUGUI textStars;
11    [SerializeField] private TextMeshProUGUI textScore;
12
13    private int scoreCoins;
14    private int scoreGems;
15    private int scoreStars;
16    private int scoreTotal;
17
18    void Awake()
19    {
20        if (instance == null) instance = this;
21    }
22
23    public void ChangeScoreCoin(int value)
24    {
25        scoreCoins += value;
26        UpdateUI();
27    }
28
29    public void ChangeScoreGem(int value)
30    {
31        scoreGems += value;
32        UpdateUI();
33    }
34
35    public void ChangeScoreStar(int value)
36    {
37        scoreStars += value;
38        UpdateUI();
39    }
40
41    private void UpdateUI()
42    {
43        scoreTotal = scoreCoins + scoreGems + scoreStars;
44        textCoins.text = scoreCoins.ToString();
45        textGems.text = scoreGems.ToString();
46        textStars.text = scoreStars.ToString();
47        textScore.text = scoreTotal.ToString();
48    }
49 }
```

Listing 3: ScoreManager.cs – Sistema de puntuación

3.4. DialogueManager – Sistema de Diálogos

Permite mostrar diálogos con NPCs, incluyendo efecto de escritura progresiva (*typewriter*).

```
1 using UnityEngine;
2 using System.Collections;
3 using System.Collections.Generic;
4 using TMPro;
5
6 public class DialogueManager : MonoBehaviour
7 {
8     public static DialogueManager instance;
9
10    [SerializeField] private TextMeshProUGUI displayText;
11    [SerializeField] private float typingSpeed;
12
13    private Queue<string> sentences = new Queue<string>();
14    private bool activeSentence;
15
16    public void StartDialogue(string[] newSentences)
17    {
18        sentences.Clear();
19        foreach (string s in newSentences)
20            sentences.Enqueue(s);
21
22        DisplayNextSentence();
23    }
24
25    public void DisplayNextSentence()
26    {
27        if (sentences.Count == 0) return;
28
29        string sentence = sentences.Dequeue();
30        StopAllCoroutines();
31        StartCoroutine(TypeSentence(sentence));
32    }
33
34    private IEnumerator TypeSentence(string sentence)
35    {
36        displayText.text = "";
37        activeSentence = true;
38        foreach (char c in sentence.ToCharArray())
39        {
40            displayText.text += c;
41            yield return new WaitForSeconds(typingSpeed);
42        }
43        activeSentence = false;
44    }
45 }
```

Listing 4: DialogueManager.cs – Diálogos con NPC

3.5. HealthBar – Barra de Vida

Actualiza visualmente la barra de vida del jugador usando un Slider.


```
1 using UnityEngine;
2 using UnityEngine.UI;
3
4 public class HealthBar : MonoBehaviour
5 {
6     [SerializeField] private Slider slider;
7
8     public void SetMaxHealth(float health)
9     {
10         slider.maxValue = health;
11         slider.value = health;
12     }
13
14     public void SetHealth(float health)
15     {
16         slider.value = health;
17     }
18 }
```

Listing 5: HealthBar.cs – UI de vida

3.6. ControlesMoviles – Joystick Virtual

Implementa controles táctiles para dispositivos móviles mediante un joystick flotante.

```
1 using UnityEngine;
2 using UnityEngine.EventSystems;
3
4 public class ControlesMoviles : MonoBehaviour,
5     IPointerDownHandler, IPointerUpHandler, IDragHandler
6 {
7     private Vector2 screenPosition;
8     private Vector2 moveByTouch;
9
10    public void OnPointerDown(PointerEventData eventData)
11    {
12        OnDrag(eventData);
13    }
14
15    public void OnDrag(PointerEventData eventData)
16    {
17        RectTransformUtility.ScreenPointToLocalPointInRectangle(
18            GetComponent<RectTransform>(),
19            eventData.position,
20            eventData.pressEventCamera,
21            out screenPosition);
22
23        moveByTouch = screenPosition.normalized;
24    }
25
26    public void OnPointerUp(PointerEventData eventData)
27    {
28        moveByTouch = Vector2.zero;
29    }
30
31    public Vector2 GetInput() => moveByTouch;
```

32 }

Listing 6: ControlesMoviles.cs – Controles táctiles

3.7. CameraController – Seguimiento de Cámara

La cámara sigue al jugador con suavizado mediante SmoothDamp.

```
1 using UnityEngine;
2
3 public class CameraController : MonoBehaviour
4 {
5     [SerializeField] private Transform target;
6     [SerializeField] private float velocidadCamara;
7
8     private Vector3 velocity = Vector3.zero;
9
10    void LateUpdate()
11    {
12        Vector3 targetPos = new Vector3(
13            target.position.x,
14            target.position.y,
15            transform.position.z);
16
17        transform.position = Vector3.SmoothDamp(
18            transform.position,
19            targetPos,
20            ref velocity,
21            velocidadCamara);
22    }
23 }
```

Listing 7: CameraController.cs – Cámara

3.8. VolumeController – Control de Audio

Controla el volumen global del juego desde el menú de opciones.

```
1 using UnityEngine;
2 using UnityEngine.Audio;
3 using UnityEngine.UI;
4
5 public class VolumenController : MonoBehaviour
6 {
7     [SerializeField] private AudioMixer audioMixer;
8     [SerializeField] private Slider slider;
9
10    void Start()
11    {
12        slider.value = PlayerPrefs.GetFloat("volume", 0.75f);
13    }
14
15    public void SetVolume(float volume)
16    {
17        audioMixer.SetFloat("AudioVolume", Mathf.Log10(volume) * 20);
18    }
19 }
```

```

18     PlayerPrefs.SetFloat("volume", volume);
19 }
20 }

```

Listing 8: VolumenController.cs – Audio

4. Mecánicas de Juego

4.1. Sistema de Combate

El jugador dispone de dos modos de ataque:

- **Espada (Sword):** Ataque cuerpo a cuerpo mediante `OverlapCircleAll` para detectar enemigos en radio.
- **Proyectil (MageBall / BossWeapon):** Instanciación de prefabs de proyectiles con trayectoria rectilínea.
- **Arco (ButtonBow):** Variante de ataque a distancia.

4.2. Sistema de Vida y Escudo

- El jugador tiene puntos de vida (*health*) representados visualmente con una `HealthBar`.
- El escudo (`shield`) reduce el daño recibido. Se implementa con el valor `shieldValue` como factor de reducción.
- El daño puede ser normal (`TakeDamage`) o verdadero (`TakeTrueDamage`), que ignora el escudo.

4.3. Recolectables y Puntuación

El sistema de puntos contempla tres tipos de objetos:

Objeto	Función	Variable
Moneda (Coin)	Puntaje básico	<code>scoreCoins</code>
Gema (Gem)	Puntaje medio	<code>scoreGems</code>
Estrella (Star)	Puntaje alto	<code>scoreStars</code>

Cuadro 2: Tipos de coleccionables

4.4. Enemigos

- **Enemy:** Enemigo estándar con patrullaje y persecución al detectar al jugador dentro de un radio.

- **Boss / BossFly / BossRun:** Jefe final con múltiples estados de comportamiento gestionados por un `StateMachineBehaviour` del Animator.

4.5. Trampas y Peligros

- **Traps:** Objetos estáticos que dañan al jugador al contacto.
- **Water:** Zona letal o de daño continuo.
- **Portal:** Transporta al jugador a otra zona o escena.

5. Interfaz de Usuario (UI)

La UI está construida con el sistema Canvas de Unity y TextMeshPro:

- **Menú principal:** Botones `BTNJugar` (jugar), opciones y salir.
- **HUD en juego:** Barra de vida, contadores de monedas, gemas y estrellas.
- **Controles móviles:** Joystick flotante (`FloatingJoystick`), botones de salto y ataque.
- **Menú de pausa:** `ResumeGame` y `QuitGame`.
- **Pantalla de victoria:** `CameraWin` y opción `PlayAgain`.
- **Pantalla de derrota:** `CameraGameOver`.
- **Diálogos:** Caja de texto con efecto typewriter.

6. Flujo del Juego

1. El jugador inicia desde el **Menú Principal** (Level 0).
2. Entra al **Level 1**, donde interactúa con NPCs mediante diálogos de tutorial.
3. Avanza por los **Levels 2 y 3**, enfrentando enemigos, esquivando trampas y recolectando objetos.
4. En el **Level 4** enfrenta al jefe final (Boss).
5. Al derrotar al jefe se muestra la **pantalla de victoria** con el puntaje total.
6. Si el jugador muere, aparece la **pantalla de derrota** con opción de reintentar.

7. Conclusiones

El videojuego **Kore** demuestra la integración de múltiples sistemas de desarrollo aprendidos durante el semestre:

- Uso del motor Unity con el pipeline URP para renderizado optimizado.
- Programación orientada a componentes en C# con el patrón *Singleton* (ScoreManager, DialogueManager).
- Implementación de física 2D, detección de colisiones y capas.
- Diseño de UI responsiva con soporte para controles táctiles.
- Gestión de estados de juego mediante el *Animator State Machine*.
- Persistencia de datos con **PlayerPrefs**.
- Arquitectura por escenas con carga dinámica mediante **SceneManager**.

El proyecto constituye un videojuego funcional y jugable que integra satisfactoriamente las herramientas trabajadas a lo largo del semestre.